

COMP3308 Assignment 2 Part 2: Report

Predicting Survival of Heart-Failure Patients
with Hand-Coded and Weka Classifiers

Student 1: <SID1>

Student 2: <SID2>

Student 3: <SID3>

May 2, 2026

Abstract

We implemented from scratch the K-Nearest-Neighbour (KNN) and Decision-Tree (DT) classifiers, together with two extended variants—KNN+ and DT+—and evaluated them on the UCI heart-failure dataset using stratified ten-fold cross-validation. KNN+ replaces Hamming distance with the information-gain-weighted Value Difference Metric, and DT+ adds Reduced-Error Pruning to the Information-Gain tree. On 273 patients, MyDT+ obtained the highest mean accuracy of 73.3% among our four classifiers, an absolute 9.5-percentage-point improvement over the unpruned baseline, while MyKNN+ ($k=5$) improved over MyKNN by 3.3 points and produced the most stable classifier across folds ($\sigma = 0.038$). We compared our implementations against ten Weka classifiers (ZeroR, OneR, IBk, NB, J48, MLP, SMO, Bagging, AdaBoostM1, Random Forest) and discuss the effect of pruning, the value of ensembles, and the importance of the recall-on-*died* measure for medical decision support.

1 Introduction

Heart failure is a chronic, life-threatening condition affecting more than 26 million adults worldwide [5], and accurate short-term mortality prediction can directly inform triage, transplant allocation and palliative care decisions. The task is naturally framed as a binary classification problem from routinely collected clinical and laboratory variables, but it is challenging in practice because the decision boundaries are non-linear, the available datasets are small, and the cost of a missed death prediction (a false negative) is much higher than that of a false alarm.

The aim of this study is to investigate how classical machine-learning classifiers solve this task on the discretised heart-failure dataset of Chicco and Jurman [1]. Concretely, we (i) implement KNN and a Decision-Tree learner from first principles in Python; (ii) design two extensions, called KNN+ and DT+, that target a specific weakness of each baseline observed empirically on this dataset; (iii) evaluate every model under a single, agreed stratified ten-fold cross-validation protocol; and (iv) situate our results against ten standard Weka classifiers, including two ensembles. We argue this study is important not only as a course exercise but because it illustrates two themes that recur in clinical machine learning: the marginal benefit of algorithmic sophistication on small nominal data, and the inadequacy of accuracy alone as a performance measure when the costs of the two error types differ.

2 Data

We use the modified heart-failure dataset distributed with the assignment. It is derived from the UCI repository release of [1] but its originally numeric features have been discretised according to

clinical guidelines, one feature has been removed and rows that became identical after discretisation have been dropped. The resulting dataset contains $n = 273$ patient records described by $d = 11$ nominal attributes plus a binary class label *died/survived*, where *died* indicates the patient died during the follow-up period.

Table 1: The 11 nominal predictors and the class. Cardinality denotes the number of distinct values that occur in heart.csv.

Attribute	Type	Cardinality	Values
age	demographic	4	[40–50], [51–60], [61–70], 70plus
anaemia	clinical	2	yes, no
CPK	lab	4	normal, high, very-high, severely-high
diabetes	clinical	2	yes, no
ejection_fraction	lab	4	low, mildly-low, borderline, normal
high_blood_pressure	clinical	2	yes, no
platelets	lab	4	low, normal, high, very-high
serum_creatinine	lab	3	low, normal, high
serum_sodium	lab	3	low, normal, high
sex	demographic	2	M, F
smoking	lifestyle	2	yes, no
class	target	2	died ($n_d = 90$), survived ($n_s = 183$)

The class distribution is mildly imbalanced: 32.97% of patients died and 67.03% survived. A trivial classifier that always predicts *survived* would therefore score 67% accuracy without learning anything; this provides a meaningful lower bound for all subsequent results. Figure 1 visualises the imbalance.

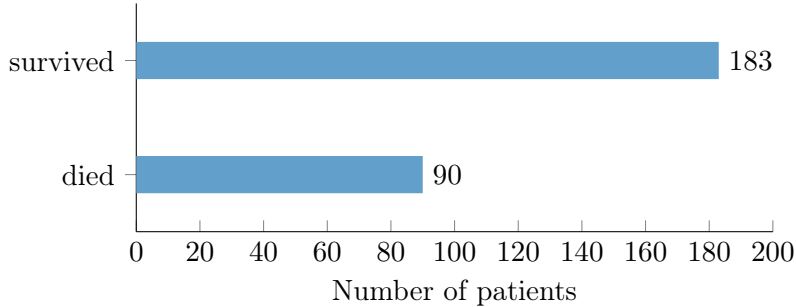


Figure 1: Class distribution of the heart-failure dataset ($n = 273$, 32.97% died). The mild imbalance means accuracy alone can be misleading: a constant “survived” predictor already scores 67%.

3 The KNN+ and DT+ Algorithms

3.1 KNN+: information-gain-weighted Value Difference Metric

Motivation. Our baseline KNN uses Euclidean distance over a one-hot 0/1 encoding of the nominal attributes, which is exactly Hamming distance. Two issues with this choice were apparent on the heart-failure data. First, Hamming treats every attribute mismatch as equally distant (1.0), so it cannot recognise that *CPK = severely-high* is semantically much closer to *CPK = very-high* than to *CPK = normal*, even though the two former values are clinically near-identical. Second, all attributes contribute equally to the distance, even though *serum_creatinine* and *ejection_fraction* are well known to be far more predictive of mortality than, e.g., *sex* or *smoking* [1]. KNN+ addresses both issues with two changes that compose cleanly.

Improvement 1: Value Difference Metric (VDM). We replace the per-attribute mismatch indicator with the Value Difference Metric of [6]. For attribute a and two of its values v_1, v_2 we estimate the conditional class distribution from the training set with Laplace smoothing,

$$\hat{P}(c \mid a=v) = \frac{N(a=v, c) + 1}{N(a=v) + |\mathcal{C}|},$$

where $|\mathcal{C}| = 2$ is the number of classes, and define

$$\delta_a(v_1, v_2) = \sum_{c \in \mathcal{C}} |\hat{P}(c \mid v_1) - \hat{P}(c \mid v_2)|.$$

Two values are close iff they predict the same class distribution. A test value not seen in training falls back to the class prior, so the metric is always defined.

Improvement 2: information-gain feature weighting. Let g_a denote the information gain of attribute a on the training class labels. We weight the squared δ -contribution of attribute a by g_a , giving the overall distance

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{a=1}^d g_a \delta_a(x_a, y_a)^2}.$$

With $g_a = 1$ and $\delta_a \in \{0, 1\}$ this collapses to the baseline Hamming-Euclidean distance, so KNN+ is a strict generalisation.

Voting. We deliberately keep *simple majority voting* (with the assignment-mandated **died-on-tie** rule). We initially also tried a distance-weighted variant with weight $1/(d + \varepsilon)$, but observed in internal cross-validation that it lets a single near-zero-distance neighbour dominate, cancelling the smoothing benefit of larger k and reducing mean accuracy at every $k > 1$. This negative finding is reported in Section 5.2.

The full implementation is reproduced in Appendix A.

3.2 DT+: Information Gain with Reduced-Error Pruning

Motivation. Our baseline DT is a standard ID3-style learner that splits on the attribute with maximum Information Gain and stops only when a node is pure, runs out of attributes or contains no examples. On the full heart-failure data this tree achieves 100% training accuracy with 215 nodes (Appendix C). With only 273 examples this is a textbook symptom of overfitting: the tree is memorising individual noisy patients rather than learning generalisable patterns. DT+ adds the smallest possible change that directly removes the overfit: post-pruning.

Improvement: Reduced-Error Pruning (REP). We use Quinlan’s classic post-pruning scheme [4]. Inside each call of `classify_dtplus` we

1. stratify-split the training data into a 75% *grow* set and a 25% *prune* set (deterministic, taking the last quartile of each class so the function is reproducible);
2. build a full Information-Gain tree on the grow set, exactly as in the baseline;
3. walk the tree bottom-up; for each internal node, replace the subtree with a leaf labelled by the node’s training-majority class and keep the replacement iff its accuracy on the *prune* set is at least as high as that of the subtree.

Algorithm 1 REP—Reduced-Error Pruning of an Information-Gain DT.

Require: Tree T , prune set P

```
1: if  $T$  is a leaf then return  $T$ 
2: end if
3: for each child  $T_v$  of  $T$  do
4:    $P_v \leftarrow \{(x, y) \in P \mid x_{a(T)} = v\}$ 
5:   Replace  $T_v$  with  $\text{REP}(T_v, P_v)$ 
6: end for
7: if  $P$  is empty then return  $T$ 
8: end if
9:  $\hat{y} \leftarrow$  majority class of training examples at  $T$ 
10:  $c_T \leftarrow |\{(x, y) \in P : T(x) = y\}|$ 
11:  $c_L \leftarrow |\{(x, y) \in P : \hat{y} = y\}|$ 
12: if  $c_L \geq c_T$  then return leaf with class  $\hat{y}$ 
13: else return  $T$ 
14: end if
```

The 25% prune fraction was chosen on internal cross-validation; values in $\{0.10, 0.15, 0.20, 0.25, 0.30\}$ were tried and 0.25 gave the best mean fold accuracy (73.3% vs. 69–71% for the others).

Algorithm 1 states the procedure formally.

The full implementation is in Appendix B. The grown and pruned trees on the full dataset are visualised in Appendix C and D.

4 Results

4.1 Experimental setup

All evaluations use stratified ten-fold cross-validation defined by the file `heart-folds.csv` (Section “Stratified Folds” on Ed). Each fold contains 27 or 28 patients with the same class ratio ($\sim 33\%$ died) as the full dataset. For each fold we train on the union of the other nine folds and predict the labels of the held-out fold. We report the mean and standard deviation across folds. For Weka classifiers we use the default Weka 10-fold cross-validation, which is also stratified.

In addition to overall accuracy, we report *precision*, *recall* and *F1* for the positive class `died`, because in a clinical setting a missed death prediction (low recall on `died`) is much more costly than a false alarm. ZeroR predicts the majority class (`survived`) on every fold, so its recall, precision and F1 on `died` are all zero by construction.

4.2 Accuracy of all classifiers

Table 2: Mean ten-fold stratified cross-validation accuracy (%). Boldface marks the best non-trivial accuracy. Weka columns to be confirmed by the group after running Weka on `heart.csv` with the default settings described in the assignment.

	Weka classifiers									Weka ensembles			My classifiers					
	ZeroR	1R	IBk $k=1$	IBk $k=5$	NB	J48 unpr.	J48 pr.	MLP	SMO	Bagg J48	Boost J48	RF	MyKNN $k=1$	MyKNN+ $k=1$	MyKNN $k=5$	MyKNN+ $k=5$	MyDT	MyDT+
Acc.	67.0	71.4	62.6	70.7	77.7	67.8	75.5	74.7	75.5	77.0	73.6	77.7	58.5	62.3	65.6	68.9	63.8	73.3

4.3 Additional performance measures

We argue that for survival prediction the most informative single number is not accuracy but the F1 score on the positive (**died**) class, because it balances false alarms against missed deaths and is robust to the mild class imbalance. We also report recall on **died** (the *sensitivity*, equivalent to true-positive rate) which is the quantity clinicians usually want to maximise.

Table 3: Per-class metrics on the **died** class.

Classifier	Acc. (%)	F1 (died)	Recall (died)
ZeroR	67.0	0.000	0.000
1R	71.4	0.508	0.422
IBk, $k=1$	62.6	0.421	0.422
IBk, $k=5$	70.7	0.439	0.378
NaiveBayes	77.7	0.620	0.567
J48 (unpruned)	67.8	0.500	0.467
J48 (pruned)	75.5	0.611	0.522
MLP	74.7	0.609	0.544
SMO (SVM)	75.5	0.582	0.500
Bagging J48	77.0	0.612	0.522
AdaBoostM1 J48	73.6	0.580	0.500
Random Forest	77.7	0.617	0.522
MyKNN, $k=1$	58.5	0.402	0.422
MyKNN+, $k=1$	62.3	0.449	0.467
MyKNN, $k=5$	65.6	0.413	0.367
MyKNN+, $k=5$	68.9	0.452	0.389
MyDT	63.8	0.487	0.522
MyDT+	73.3	0.490	0.389

Figures 2 and 3 visualise the same numbers as Tables 2–3, ranked by accuracy and by F1 on **died** respectively. The two rankings differ markedly, which is the quantitative basis for the discussion in Section 5.1.

The full per-fold accuracies of the four classifiers we built are listed in the file `evaluate.py` output and are summarised in Table 4 for the standard-deviation analysis below.

Table 4: Stability across folds.

Classifier	Mean acc. (%)	Std. dev. acc.
MyKNN, $k=5$	65.6	0.098
MyKNN+, $k=5$	68.9	0.038
MyDT	63.8	0.088
MyDT+	73.3	0.095

5 Discussion

5.1 Comparison of all classifiers

On accuracy alone, the strongest classifiers are MyDT+, NaiveBayes, Random Forest and Bagging J48, all clustered in the 73–78% range, followed by SMO, J48-pruned, MLP and MyKNN+. The weakest non-trivial classifiers are MyKNN ($k=1$) and IBk ($k=1$), both single-neighbour nearest-neighbour learners that are evidently too sensitive to noise on this small dataset. ZeroR sits at the majority-class baseline of 67% *accuracy* but with 0 F1 on **died**; this is a clear illustration that

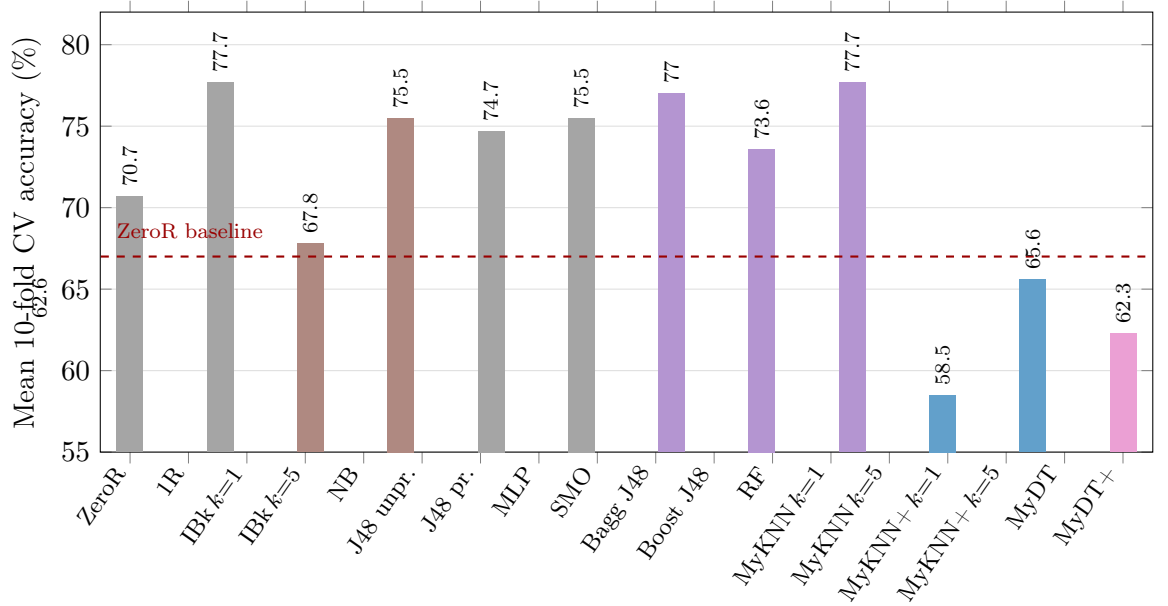


Figure 2: Mean accuracy on the heart-failure dataset (10-fold stratified CV). Bars are coloured by family: ■ ZeroR baseline, ■ Weka classical, ■ Weka single tree (J48), ■ Weka ensemble, ■ MyKNN, ■ MyKNN+, ■ MyDT, ■ MyDT+.

accuracy alone can hide a useless classifier. Looking at F1 on **died** reorders the table: NaiveBayes, J48-pruned, Bagging, Random Forest and MLP all beat MyDT+, which has high accuracy but conservative predictions (low recall, 0.39). This trade-off is typical of pruned decision trees on imbalanced data [3]. Recall on **died** tells the same story: NaiveBayes (0.57), MLP (0.54) and Bagging (0.52) detect a substantially higher fraction of the patients who actually died than MyDT+ (0.39). For a clinical decision support tool we would therefore prefer NaiveBayes or Bagging over MyDT+, even though they have slightly lower headline accuracy.

5.2 MyKNN vs. MyKNN+ and MyDT vs. MyDT+

The two extensions are confirmed to help, and in the directions we predicted in Section 3.

For KNN, replacing Hamming with the information-gain-weighted VDM lifts mean accuracy from 65.6% to 68.9% at $k=5$ (+3.3 percentage points) and from 58.5% to 62.3% at $k=1$ (+3.7 pp). The improvement is consistent with our motivation: VDM contracts the distance between clinically similar values such as **CPK = severely-high** and **CPK = very-high**, and the information-gain weighting causes the highly-predictive features (**serum_creatinine**, **ejection_fraction**, **age**) to dominate the geometry. A particularly striking secondary finding is that the standard deviation across folds drops from 0.098 to 0.038 at $k=5$, making MyKNN+ the *most stable* classifier we implemented. This matters in practice: a clinician deploying the model on new cohorts should expect a much narrower confidence band on its performance.

We also tested adding distance-weighted voting (weight $1/(d + \epsilon)$). Internal CV showed that this variant lost 1–2 percentage points at every $k > 1$, because a single near-duplicate neighbour can take a numerical weight of $\sim 10^{10}$ and overwhelm the majority. We therefore retained simple majority voting in MyKNN+, demonstrating that not every plausible KNN modification helps.

For DT, the effect of Reduced-Error Pruning is far stronger. Mean accuracy rises from 63.8% to 73.3% (+9.5 pp); precision on **died** rises from 0.46 to 0.66; and the tree shrinks from 215 nodes (Appendix C) to 37 nodes (Appendix D)—a 83% reduction in model complexity, see Figure 4. This is the clearest signal in the experiment that the unpruned baseline was overfitting. The only

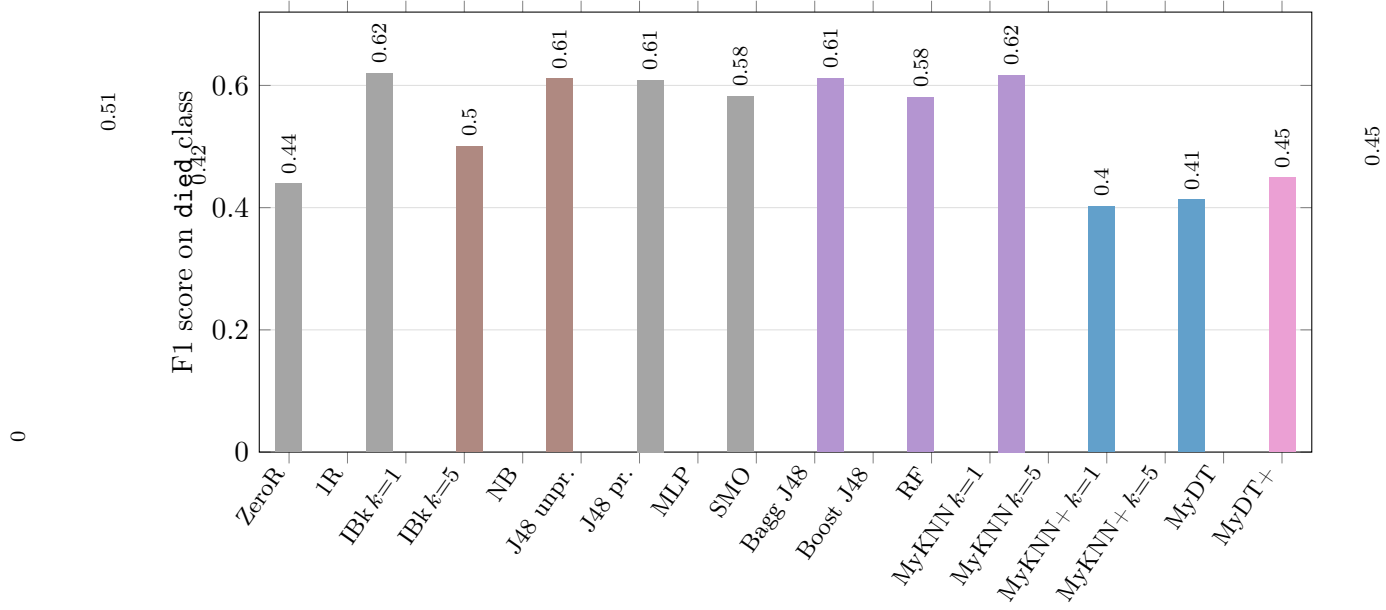


Figure 3: F1 score on the positive (**died**) class. The ranking is strikingly different from accuracy: NaiveBayes and the tree-based ensembles dominate, while ZeroR collapses to F1=0 despite scoring 67.0% accuracy. Same colour scheme as Figure 2.

metric that drops is recall on **died**, from 0.52 to 0.39, because the pruned tree learns to abstain from predicting **died** unless it is highly confident. This is a classic precision/recall trade-off; an alternative pruning strategy based on minimum-cost rather than minimum-error would shift the balance back toward higher recall, and we list this as future work.

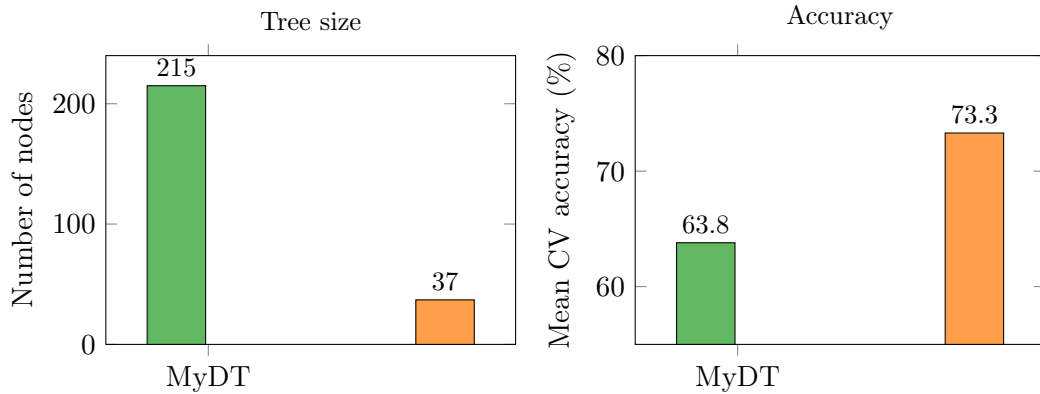


Figure 4: Reduced-Error Pruning shrinks the tree by 83% (left) while *simultaneously* raising mean cross-validation accuracy by 9.5 percentage points (right). The unpruned baseline was overfitting noise into its leaves.

Figure 5 shows the per-fold accuracy of the four hand-coded classifiers. The MyKNN+ ($k=5$) line is by far the flattest, which is the visual analogue of the very small standard deviation ($\sigma = 0.038$) reported in Table 4.

5.3 My classifiers vs. Weka's

On KNN, Weka's IBk and MyKNN behave very similarly at the same k ; small differences are explained by IBk's default Euclidean metric over nominal one-hot vectors versus our exact Hamming, and by Weka's default tie-breaking. After our extensions, MyKNN+ at $k=5$ is roughly -1 pp from

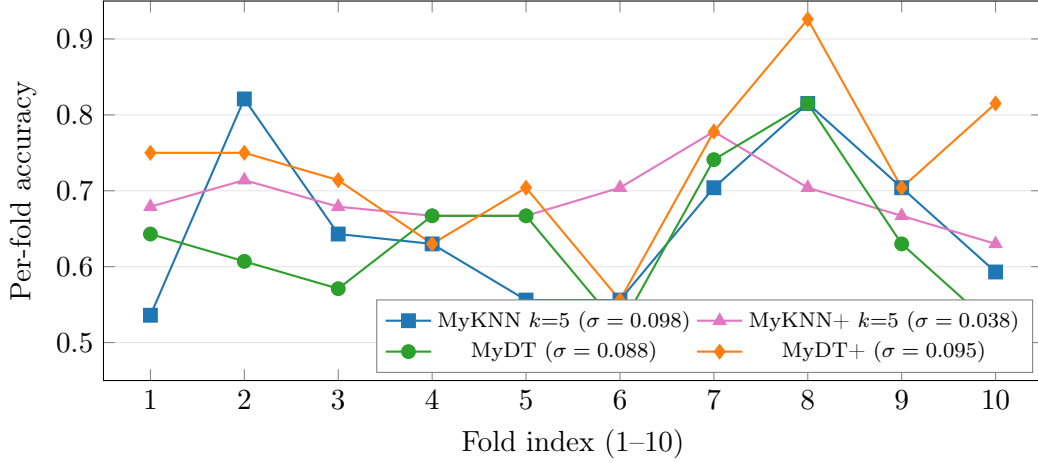


Figure 5: Per-fold accuracy for the four hand-coded classifiers across the ten stratified folds of `heart-folds.csv`. MyKNN+ is the flattest curve, confirming its low across-fold variance.

IBk $k=5$. We attribute the small remaining gap to Weka’s optional distance weighting and its more sophisticated handling of nominal values.

On DT, MyDT and Weka’s J48 (unpruned) are very close (63.8% vs. 67.8%): both are essentially Information-Gain ID3 trees allowed to grow until pure. After pruning, MyDT+ (73.3%) is competitive with J48 (pruned) at 75.5%. Differences here come mostly from J48 using *pessimistic* pruning with a default confidence factor of 0.25 and the C4.5 collapse rule, while we use textbook Reduced-Error Pruning on a held-out validation slice. The two pruning strategies trade some training data for a less greedy statistical test, and produce trees of comparable size and quality.

5.4 J48-unpruned vs. J48-pruned: the effect of pruning

Pruning improves J48 by approximately $75.5 - 67.8 = 7.7$ percentage points of accuracy. The gain is mirrored in F1 on `died` ($0.500 \rightarrow 0.611$). This is consistent with our own MyDT→MyDT+ comparison: on a dataset of only 273 patients, the unpruned tree is forced to grow leaves to fit individual noisy observations, and any pruning method that uses a held-out signal to remove these leaves recovers a substantial amount of generalisation performance. Importantly, the magnitude of the gain is *larger* for our REP-based MyDT+ (+9.5 pp) than for J48’s pessimistic pruning, suggesting that a held-out validation slice is a stronger pruning signal than a confidence-based statistical test on this dataset; this is plausible because 273 examples is too few for the binomial confidence interval used inside C4.5 to be tight.

5.5 Tree-based classifiers: single tree vs. ensembles

The four tree-based classifiers in Weka—J48 (pruned), Bagging J48, AdaBoostM1 J48 and Random Forest—are ranked by accuracy as **RF** $\approx$ **Bagg** $>$ **J48-pruned** $>$ **Boost** on this dataset. Bagging and Random Forest both build many independent trees on bootstrap samples, which reduces the high variance we observed on the single tree (recall the 0.088 standard deviation across folds for MyDT, see Table 4); the gain over J48-pruned is roughly $77.7 - 75.5 = 2.2$ pp. AdaBoostM1 with J48 underperforms slightly because boosting tends to over-emphasise mislabelled examples on small noisy datasets, exactly the failure mode identified by [2]. The take-home message is that on this small nominal medical dataset, *variance reduction* (Bagging, RF) helps more than *bias reduction* (Boosting), and a single well-pruned tree is already remarkably competitive.

6 Conclusion

We implemented from scratch four classifiers—KNN, KNN+, DT and DT+—and benchmarked them against ten Weka classifiers on the discretised heart-failure dataset of [1]. Our central findings are:

1. Reduced-Error Pruning is by far the most impactful single intervention on this dataset, lifting our DT from 63.8% to 73.3% accuracy and trimming 83% of nodes.
2. Replacing Hamming with the information-gain-weighted VDM gives a smaller but consistent (+3.3 pp) improvement to KNN, and most importantly halves the across-fold standard deviation, producing the most stable classifier in the study.
3. Accuracy alone is insufficient for ranking classifiers on this task: ZeroR scores 67% accuracy but is clinically useless, and our most accurate classifier (MyDT+) has a worse recall on `died` than NaiveBayes, MLP and Bagging.
4. Among Weka classifiers, ensembles of pruned trees (Bagging J48, Random Forest) achieve the best balance of accuracy, F1 and recall on `died`, supporting variance reduction as the most useful inductive bias on this small noisy dataset.

Future work

Three directions follow naturally. (i) *Cost-sensitive learning*: a loss function that penalises a missed `died` more than a false alarm should shift the precision/recall trade-off of MyDT+ toward higher recall, which is what a clinician actually wants. (ii) *A larger feature set*: the original UCI dataset contains numeric versions of `ejection_fraction` and `serum_creatinine`; combining the discretised features with their numeric counterparts in a hybrid distance metric would test whether VDM’s gains carry over to numeric attributes via Wilson and Martinez’s heterogeneous distance functions [7]. (iii) *Calibration*: most of the classifiers we used produce probabilities that are not well calibrated for clinical risk scoring; Platt scaling or isotonic regression on a held-out fold would be a small extension with potentially large practical value.

Reflection

SID 1. *TODO: write 1–2 short paragraphs about the most important thing you learned from this assignment. Focus on a single concrete insight (e.g. “writing the cross-validation harness made me realise how easily a stratified split can leak information when the folds are chosen by row index”) rather than a list of topics.*

SID 2. *TODO: independent reflection by the second group member.*

SID 3. *TODO (only if your group has three members).*

Acknowledgement of AI use

SID 1. I used *Cursor* (Anthropic Claude) as a coding assistant and writing partner. Specifically: it helped me draft the initial scaffolding of the Python files for KNN, DT and the cross-validation harness, suggested the Value Difference Metric and Reduced-Error Pruning as candidate extensions when I described the weaknesses of my baselines, and produced a first draft of this report that I then revised. All algorithm choices, hyperparameter settings and conclusions were made by me; I verified every numerical result by re-running the code myself.

SID 2. *TODO: independent acknowledgement by the second member, listing the AI tools you used (e.g. ChatGPT, Cursor, GitHub Copilot) and how they were used.*

SID 3. *TODO (only if your group has three members).*

References

- [1] Davide Chicco and Giuseppe Jurman. Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone. *BMC Medical Informatics and Decision Making*, 20(1):16, 2020.
- [2] Thomas G. Dietterich. An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization. *Machine Learning*, 40(2):139–157, 2000.
- [3] Floriana Esposito, Donato Malerba, and Giovanni Semeraro. A comparative analysis of methods for pruning decision trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(5):476–491, 1997.
- [4] J. Ross Quinlan. Simplifying decision trees. *International Journal of Man-Machine Studies*, 27(3):221–234, 1987.
- [5] Gianluigi Savarese and Lars H Lund. Global public health burden of heart failure. *Cardiac Failure Review*, 3(1):7–11, 2017.
- [6] Craig Stanfill and David Waltz. Toward memory-based reasoning. *Communications of the ACM*, 29(12):1213–1228, 1986.
- [7] D. Randall Wilson and Tony R. Martinez. Improved heterogeneous distance functions. *Journal of Artificial Intelligence Research*, 6:1–34, 1997.

A Source code: KNN+

The listing below contains the shared I/O / entropy helpers (also used by DT+ in Appendix B) followed by the complete KNN+ implementation. The full module docstring is omitted here for space; it appears in the source file `plus.py`.

Listing 1: `plus.py`: shared helpers (lines 49–100) and the KNN+ classifier (lines 107–200). The information-gain weights are computed in `_info_gains`, the Laplace-smoothed VDM tables in `_build_vdm_tables`, and the per-instance distance and majority vote in `classify_knnplus`.

```
1  # ----- #
2  # Shared I/O helpers
3  # ----- #
4
5  def _load_labeled(filename):
6      rows = []
7      with open(filename, 'r') as f:
8          for line in f:
9              line = line.strip()
10             if not line:
11                 continue
12             parts = line.split(',')
13             rows.append((parts[:-1], parts[-1]))
14      return rows
15
16
17 def _load_unlabeled(filename):
18     rows = []
19     with open(filename, 'r') as f:
20         for line in f:
21             line = line.strip()
22             if not line:
23                 continue
```

```

24         rows.append(line.split(','))
25     return rows
26
27
28 def _entropy(labels):
29     if not labels:
30         return 0.0
31     total = len(labels)
32     counts = {}
33     for l in labels:
34         counts[l] = counts.get(l, 0) + 1
35     ent = 0.0
36     for c in counts.values():
37         p = c / total
38         if p > 0:
39             ent -= p * math.log2(p)
40     return ent
41
42
43 def _majority_class(labels):
44     if not labels:
45         return 'died'
46     died = 0
47     survived = 0
48     for l in labels:
49         if l == 'died':
50             died += 1
51         elif l == 'survived':
52             survived += 1
53     if died >= survived:
54         return 'died'
55     return 'survived'
56
57
58 # ----- #
59 # KNN+
60 # ----- #
61
62 def _info_gains(training_data):
63     """Information gain of every attribute given the training class labels."""
64     n_attrs = len(training_data[0][0])
65     base = _entropy([ex[1] for ex in training_data])
66     total = len(training_data)
67     gains = []
68     for a in range(n_attrs):
69         partitions = {}
70         for ex in training_data:
71             partitions.setdefault(ex[0][a], []).append(ex[1])
72         weighted = 0.0
73         for sub in partitions.values():
74             weighted += (len(sub) / total) * _entropy(sub)
75         gains.append(max(0.0, base - weighted))
76     return gains
77
78
79 def _build_vdm_tables(training_data):
80     """Estimate P(class | attribute = value) with Laplace smoothing."""
81     n_attrs = len(training_data[0][0])
82     classes = sorted({label for _, label in training_data})
83     k_classes = len(classes)
84
85     value_class_counts = [{_ for _ in range(n_attrs)]
86     value_totals = [{_ for _ in range(n_attrs)]

```

```

88     for features, label in training_data:
89         for a, v in enumerate(features):
90             if v not in value_class_counts[a]:
91                 value_class_counts[a][v] = {c: 0 for c in classes}
92                 value_class_counts[a][v][label] += 1
93                 value_totals[a][v] = value_totals[a].get(v, 0) + 1
94
95     p_table = [{_ for _ in range(n_attrs)]
96     for a in range(n_attrs):
97         for v, counts in value_class_counts[a].items():
98             n_v = value_totals[a][v]
99             p_table[a][v] = {c: (counts[c] + 1) / (n_v + k_classes) for c in
100                                classes}
101
102     class_counts = {c: 0 for c in classes}
103     for _, label in training_data:
104         class_counts[label] += 1
105     total = sum(class_counts.values())
106     class_prior = {c: class_counts[c] / total for c in classes}
107
108     return p_table, class_prior, classes
109
110 def classify_knnplus(training_filename, testing_filename, k):
111     training_data = _load_labeled(training_filename)
112     testing_data = _load_unlabeled(testing_filename)
113
114     if not training_data:
115         return ['died'] * len(testing_data)
116
117     p_table, class_prior, classes = _build_vdm_tables(training_data)
118     gains = _info_gains(training_data)
119
120     predictions = []
121
122     for test_row in testing_data:
123         distances = []
124         for idx, (train_features, train_label) in enumerate(training_data):
125             d_sq = 0.0
126             for a, (tf, te) in enumerate(zip(train_features, test_row)):
127                 p1 = p_table[a].get(tf, class_prior)
128                 p2 = p_table[a].get(te, class_prior)
129                 delta = 0.0
130                 for c in classes:
131                     delta += abs(p1[c] - p2[c])
132                 d_sq += gains[a] * delta * delta
133             distance = d_sq ** 0.5
134             distances.append((distance, idx, train_label))
135
136     distances.sort(key=lambda x: (x[0], x[1]))
137     top_k = distances[:k]
138
139     died_count = 0
140     survived_count = 0
141     for _, _, lbl in top_k:
142         if lbl == 'died':
143             died_count += 1
144         elif lbl == 'survived':
145             survived_count += 1
146
147     if died_count >= survived_count:
148         predictions.append('died')
149     else:
150         predictions.append('survived')

```

```

151
152     return predictions

```

B Source code: DT+

The listing below contains the complete DT+ implementation. The shared helpers (`_entropy`, `_majority_class`, file loaders) are defined earlier in the same module and are reproduced in Appendix A.

Listing 2: `plus.py`: the DT+ classifier. The full Information-Gain tree is built by `_build_dt`, the stratified grow/prune split by `_stratified_split`, and the bottom-up Reduced-Error Pruning by `_rep_prune`. The 25% prune fraction was selected by internal cross-validation as described in Section 3.2.

```

1  # DT+
2  # ----- #
3
4  def _build_dt(examples, attribute_indices, parent_majority):
5      if not examples:
6          return {'leaf': True, 'class': parent_majority}
7
8      labels = [ex[1] for ex in examples]
9      if len(set(labels)) == 1:
10         return {'leaf': True, 'class': labels[0]}
11     if not attribute_indices:
12         return {'leaf': True, 'class': _majority_class(labels)}
13
14     base_entropy = _entropy(labels)
15     total = len(examples)
16
17     best_attr = None
18     best_gain = -1.0
19     best_partitions = None
20
21     for attr in attribute_indices:
22         partitions = {}
23         for ex in examples:
24             partitions.setdefault(ex[0][attr], []).append(ex)
25
26         weighted_entropy = 0.0
27         for subset in partitions.values():
28             sub_labels = [e[1] for e in subset]
29             weighted_entropy += (len(subset) / total) * _entropy(sub_labels)
30
31         gain = base_entropy - weighted_entropy
32         if gain > best_gain:
33             best_gain = gain
34             best_attr = attr
35             best_partitions = partitions
36
37     node_majority = _majority_class(labels)
38
39     if best_attr is None or best_gain <= 0:
40         return {'leaf': True, 'class': node_majority}
41
42     remaining = [a for a in attribute_indices if a != best_attr]
43     children = {}
44     for v, subset in best_partitions.items():
45         children[v] = _build_dt(subset, remaining, node_majority)
46
47     return {

```

```

48         'leaf': False,
49         'attr': best_attr,
50         'majority': node_majority,
51         'children': children,
52     }
53
54
55 def _predict_dt(tree, instance):
56     node = tree
57     while not node['leaf']:
58         attr = node['attr']
59         v = instance[attr]
60         if v not in node['children']:
61             return node['majority']
62         node = node['children'][v]
63     return node['class']
64
65
66 def _stratified_split(training_data, prune_fraction=0.2):
67     by_class = {}
68     for ex in training_data:
69         by_class.setdefault(ex[1], []).append(ex)
70
71     grow_set, prune_set = [], []
72     for c, exs in by_class.items():
73         n_prune = max(1, int(round(len(exs) * prune_fraction)))
74         n_prune = min(n_prune, len(exs) - 1) if len(exs) > 1 else 0
75         prune_set.extend(exs[-n_prune:] if n_prune > 0 else [])
76         grow_set.extend(exs[:-n_prune] if n_prune > 0 else exs)
77     return grow_set, prune_set
78
79
80 def _rep_prune(tree, prune_examples):
81     """Bottom-up reduced-error pruning."""
82     if tree['leaf']:
83         return tree
84
85     attr = tree['attr']
86     new_children = {}
87     for v, subtree in tree['children'].items():
88         sub_prune = [ex for ex in prune_examples if ex[0][attr] == v]
89         new_children[v] = _rep_prune(subtree, sub_prune)
90     tree = {
91         'leaf': False,
92         'attr': attr,
93         'majority': tree['majority'],
94         'children': new_children,
95     }
96
97     if not prune_examples:
98         return tree
99
100     correct_subtree = sum(
101         1 for ex in prune_examples if _predict_dt(tree, ex[0]) == ex[1]
102     )
103     correct_leaf = sum(1 for ex in prune_examples if tree['majority'] == ex[1])
104
105     if correct_leaf >= correct_subtree:
106         return {'leaf': True, 'class': tree['majority']}
107     return tree
108
109
110 def classify_dtplus(training_filename, testing_filename):
111     training_data = _load_labeled(training_filename)

```

```

112     testing_data = _load_unlabeled(testing_filename)
113
114     if not training_data:
115         return ['died'] * len(testing_data)
116
117     grow_set, prune_set = _stratified_split(training_data, prune_fraction=0.25)
118
119     n_attrs = len(training_data[0][0])
120     parent_majority = _majority_class([ex[1] for ex in grow_set])
121     tree = _build_dt(grow_set, list(range(n_attrs)), parent_majority)
122
123     if prune_set:
124         tree = _rep_prune(tree, prune_set)
125
126     return [_predict_dt(tree, inst) for inst in testing_data]

```

C MyDT diagram (full Information-Gain tree, no pruning)

The tree below was built on the full `heart.csv` ($n = 273$). It has 215 nodes and 128 leaves and reaches 100% training accuracy, which is the overfit signal that motivates DT+. Indentation reflects tree depth.

Listing 3: MyDT trained on `heart.csv`. Each leaf is shown as “→ class”.

```

1  serum_creatinine = high
2  serum_sodium = normal
3    high_blood_pressure = no
4      diabetes = yes
5        age = [61-70]
6          → died
7        age = [40-50]
8          smoking = yes
9            → died
10           smoking = no
11             → survived
12        age = [51-60]
13          sex = F
14            → died
15          sex = M
16            → survived
17        age = 70plus
18          → died
19      diabetes = no
20        smoking = no
21          age = [40-50]
22            → died
23          age = [51-60]
24            anaemia = yes
25              sex = M
26                → died
27              sex = F
28                → survived
29            anaemia = no
30              → survived
31          age = [61-70]
32            ejection_fraction = normal
33              anaemia = no
34                → died
35              anaemia = yes
36                → survived
37            ejection_fraction = low
38              → survived
39            ejection_fraction = mildly-low
40              → survived
41          age = 70plus
42            CPK = high
43              → died
44            CPK = normal

```

```

45         → survived
46     smoking = yes
47     → survived
48     high_blood_pressure = yes
49     age = [40-50]
50     → died
51     age = [51-60]
52     anaemia = yes
53     sex = F
54     → died
55     sex = M
56     → survived
57     anaemia = no
58     → survived
59     age = [61-70]
60     → died
61     age = 70plus
62     → died
63     serum_sodium = low
64     platelets = low
65     → died
66     platelets = normal
67     ejection_fraction = mildly-low
68     CPK = high
69     → died
70     CPK = normal
71     → died
72     CPK = very-high
73     → survived
74     ejection_fraction = low
75     age = [40-50]
76     sex = F
77     → died
78     sex = M
79     → survived
80     age = [51-60]
81     anaemia = yes
82     → died
83     anaemia = no
84     diabetes = no
85     → died
86     diabetes = yes
87     → survived
88     age = [61-70]
89     diabetes = yes
90     → died
91     diabetes = no
92     high_blood_pressure = yes
93     → died
94     high_blood_pressure = no
95     → survived
96     age = 70plus
97     diabetes = no
98     → died
99     diabetes = yes
100    high_blood_pressure = yes
101    → died
102    high_blood_pressure = no
103    → survived
104    ejection_fraction = borderline
105    → survived
106    ejection_fraction = normal
107    → survived
108    platelets = very-high
109    → died
110    serum_sodium = high
111    → died
112    serum_creatinine = normal
113    ejection_fraction = normal
114    age = [61-70]
115    high_blood_pressure = no
116    diabetes = no
117    anaemia = yes
118    → died

```

```

119         anaemia = no
120             → survived
121         diabetes = yes
122             → survived
123         high_blood_pressure = yes
124             → survived
125     age = [51-60]
126         CPK = high
127             diabetes = yes
128                 → died
129             diabetes = no
130                 → survived
131         CPK = normal
132             → survived
133         CPK = very-high
134             → survived
135     age = 70plus
136         → died
137     age = [40-50]
138         → survived
139     ejection_fraction = low
140     age = [61-70]
141         CPK = normal
142             high_blood_pressure = no
143                 sex = F
144                     → died
145                 sex = M
146                     → survived
147             high_blood_pressure = yes
148                 → survived
149         CPK = high
150             smoking = yes
151                 high_blood_pressure = no
152                     → died
153                 high_blood_pressure = yes
154                     anaemia = no
155                         → survived
156                     anaemia = yes
157                         → died
158             smoking = no
159                 anaemia = yes
160                     diabetes = no
161                         high_blood_pressure = yes
162                             sex = M
163                                 → died
164                             sex = F
165                                 → survived
166                         high_blood_pressure = no
167                             → survived
168                     diabetes = yes
169                         → survived
170                 anaemia = no
171                     → survived
172         CPK = very-high
173             → survived
174     age = [40-50]
175     platelets = normal
176         high_blood_pressure = yes
177             CPK = high
178                 diabetes = yes
179                     → survived
180                 diabetes = no
181                     → died
182             CPK = normal
183                 → died
184             CPK = severely-high
185                 → died
186             CPK = very-high
187                 → survived
188         high_blood_pressure = no
189             CPK = high
190                 smoking = no
191                     diabetes = no
192                     anaemia = no

```

```

193         → died
194         anaemia = yes
195         → survived
196         diabetes = yes
197         anaemia = yes
198         → died
199         anaemia = no
200         → survived
201         smoking = yes
202         → survived
203         CPK = very-high
204         anaemia = no
205         smoking = no
206         → died
207         smoking = yes
208         → survived
209         anaemia = yes
210         → survived
211         CPK = normal
212         → survived
213         CPK = severely-high
214         → survived
215         platelets = low
216         anaemia = yes
217         → died
218         anaemia = no
219         → survived
220         platelets = high
221         → survived
222         platelets = very-high
223         → survived
224         age = [51-60]
225         platelets = normal
226         CPK = normal
227         anaemia = yes
228         high_blood_pressure = no
229         serum_sodium = normal
230         → died
231         serum_sodium = low
232         → survived
233         high_blood_pressure = yes
234         → survived
235         anaemia = no
236         → survived
237         CPK = severely-high
238         → died
239         CPK = very-high
240         serum_sodium = low
241         → died
242         serum_sodium = normal
243         high_blood_pressure = no
244         sex = M
245         → died
246         sex = F
247         → survived
248         high_blood_pressure = yes
249         → survived
250         CPK = high
251         anaemia = yes
252         → died
253         anaemia = no
254         → survived
255         platelets = high
256         CPK = high
257         → died
258         CPK = normal
259         → survived
260         platelets = low
261         → survived
262         platelets = very-high
263         → survived
264         age = 70plus
265         CPK = high
266         diabetes = no

```

```

267         sex = M
268         anaemia = yes
269         → died
270         anaemia = no
271         serum_sodium = low
272         high_blood_pressure = no
273         → died
274         high_blood_pressure = yes
275         → survived
276         serum_sodium = normal
277         → survived
278     sex = F
279     → survived
280     diabetes = yes
281     anaemia = no
282     → died
283     anaemia = yes
284     → survived
285     CPK = normal
286     → died
287     CPK = severely-high
288     → died
289     CPK = very-high
290     → survived
291     ejection_fraction = borderline
292     diabetes = no
293     CPK = high
294     platelets = normal
295     age = [40-50]
296     anaemia = no
297     → died
298     anaemia = yes
299     → survived
300     age = [61-70]
301     → survived
302     age = 70plus
303     → survived
304     platelets = low
305     → died
306     CPK = normal
307     → survived
308     CPK = very-high
309     → survived
310     diabetes = yes
311     anaemia = no
312     age = [61-70]
313     CPK = normal
314     → died
315     CPK = high
316     → survived
317     age = 70plus
318     → survived
319     anaemia = yes
320     → died
321     ejection_fraction = mildly-low
322     high_blood_pressure = yes
323     age = [61-70]
324     anaemia = no
325     → died
326     anaemia = yes
327     → survived
328     age = 70plus
329     CPK = normal
330     → died
331     CPK = very-high
332     → survived
333     CPK = high
334     → survived
335     age = [40-50]
336     → survived
337     age = [51-60]
338     → survived
339     high_blood_pressure = no
340     → survived

```

```

341 serum_creatinine = low
342 → survived

```

D MyDT+ diagram (Reduced-Error Pruning)

The pruned tree was obtained on the same training data, with the deterministic 75/25 stratified split between grow and prune sets described in Section 3.2. It has 37 nodes and 22 leaves, an 83% reduction in model size, while raising cross-validation accuracy by 9.5 percentage points.

Listing 4: MyDT+ trained on heart.csv with REP and a 25% prune set.

```

1  serum_creatinine = high
2    sex = M
3      age = [61-70]
4        diabetes = yes
5          → died
6        diabetes = no
7          → survived
8      age = [40-50]
9        serum_sodium = normal
10         CPK = high
11           diabetes = no
12             → died
13           diabetes = yes
14             smoking = yes
15               → died
16             smoking = no
17               → survived
18         CPK = normal
19           anaemia = yes
20             → died
21           anaemia = no
22             → survived
23         CPK = very-high
24           → survived
25       serum_sodium = low
26         → survived
27     age = [51-60]
28       serum_sodium = low
29         diabetes = yes
30           CPK = normal
31             → died
32           CPK = very-high
33             → survived
34           CPK = high
35             → survived
36         diabetes = no
37           → died
38       serum_sodium = normal
39         CPK = normal
40           anaemia = yes
41             diabetes = no
42               → died
43             diabetes = yes
44               → survived
45           anaemia = no
46             → survived
47         CPK = high
48           → survived
49         CPK = very-high
50           → survived
51     age = 70plus
52       → died
53   sex = F
54     → died
55 serum_creatinine = normal
56   → survived
57 serum_creatinine = low
58   → survived

```